WOOJIN LIM 301435931

Memory Locality and Branch Predictability in BFS: An Analysis of Data Structures and SIMD Optimization

Breadth-First-Search(BFS) is a fundamental graph traversal algorithm. It explores all the vertices in the graph on current depth before moving on to the next level of depth. This project explores the efficiency and memory locality of BFS implementation using different data structures: queue,deque and linked list and SIMD(single instruction, multiple data) instruction: deque, and deque with SIMD.

Included file with this paper:

new2queue.cpp, new2deque.cpp, new2ll.cpp, new2simd.cpp, new2queue.h, new2deque.h, new2ll.h, new2simd.h, vectorclass library by Agner Fog.

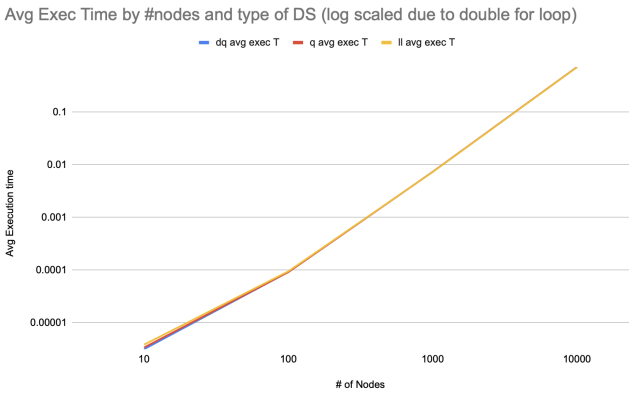
In every cpp file, researchers can manipulate the # of the nodes in main().

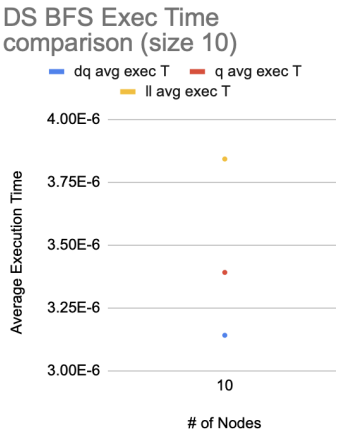
Things to Test

1. Testing Execution Time of BFS with different types of data structures.
    a. Queue vs Deque vs Linked List
2. Testing SIMD instruction on Deque based BFS
    a. Deque vs SIMD instruction Deque
3. Check Memory locality (with Deque BFS)
    a. # of nodes (10, 100, 1000, 10000)
    b. Check regression lines (X = # of nodes, Y= Execution time, cache-miss percentage)
4. Branch Predictability (with Deque BFS, # of Node = 10000)
    a. Random graph vs Linear graph

1. Testing Efficiency of BFS with different types of data structures.

In this experiment, I want to know which data structure has the shortest execution time. I used chrono to measure time. All the graphs are random graphs so reduce the error by the random, I tested 10 times by size and by data structure.





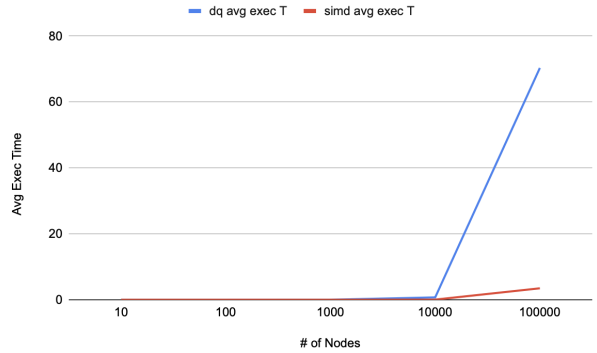
	10	100	1000	10000
deque	0.0000031412	0.0000915683	0.007295474	0.7101527
queue	0.0000033909	0.0000911877	0.007305225	0.712286
linked list	0.0000038413	0.0000945236	0.007259235	0.70643
	(First row = # of nodes)		(Numbers = Avg Exec time)	

As a test result, Deque is the most efficient data structure for the BFS implementation. And Linked List was the worst data structure(# of node = 10). When the number of nodes becomes 10000, linked lists become the fastest data structure of the BFS implementation.

2. Testing SIMD instruction on Deque based BFS

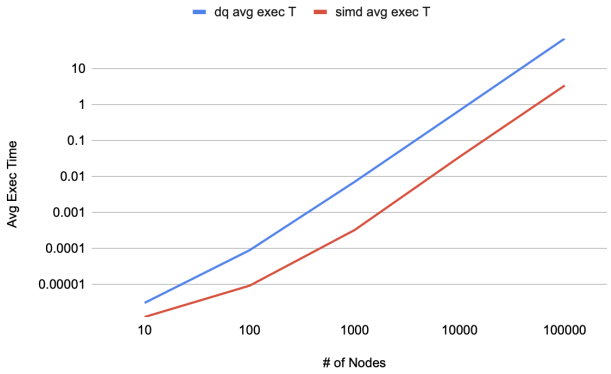
Before I started the experiment, I expected that SIMD instruction does not impact too much on execution time. Because, BFS' execution time has much more weight on branch prediction quality.

The reason why I choose deque as an experiment model is because deque is much more flexible to push and pop the data.



	10	100	1000	10000	100000
Deque	0.0000031412	0.0000915683	0.007295474	0.7101527	70.3469
SIMD Deque	0.0000012703	0.0000094526	0.0003335608	0.03636798	3.47051
	(First row = # of nodes)		(Numbers = Avg Exec time)		

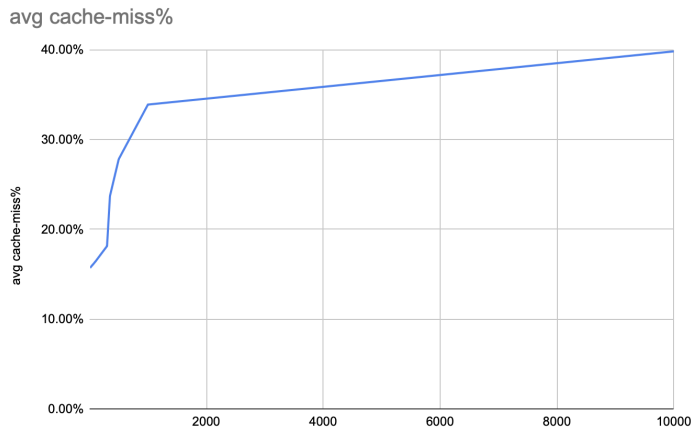
Surprisingly, SIMD BFS was x2 to x23 faster than Deque BFS. The only difference between two was SIMD used <vector> on visited list and bfs_order list. I am also curious about the general execution time per node. So I use the log scaling on the vertical axis.

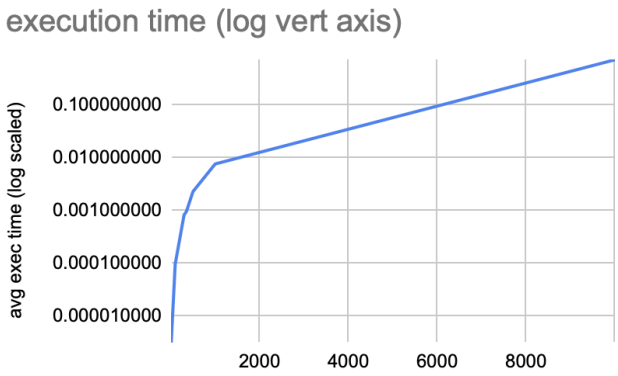


This graph is a log scaled graph, it shows that none-size biased average execution time. As we can see the more nodes the longer time each node takes. Which is natural because each node takes around 10 times more neighbor nodes. Anyway, SIMD is still effective in every node size group.

3. Checking Memory Locality

In this experiment, I will use perf to measure the cache-reference and cache-miss to check the Memory Locality. I thought the # of the nodes matter on the cache-miss because the more # of nodes the more memory required which leads us to cache-miss. To justify this hypothesis, the execution time was recorded 10 times for each node size and visualized as line graphs (X = #of nodes ,Y = cache-misses percentage & X = # of nodes, Y = execution time(log scaled)).

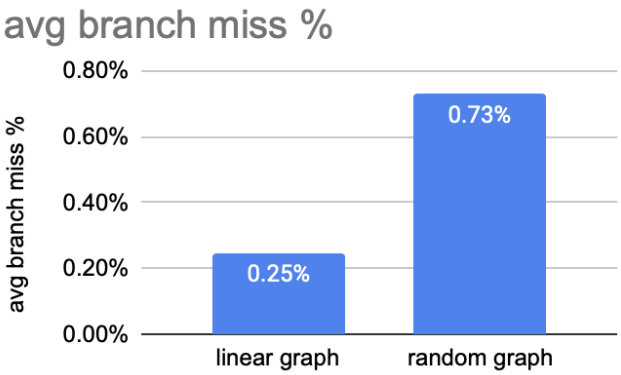




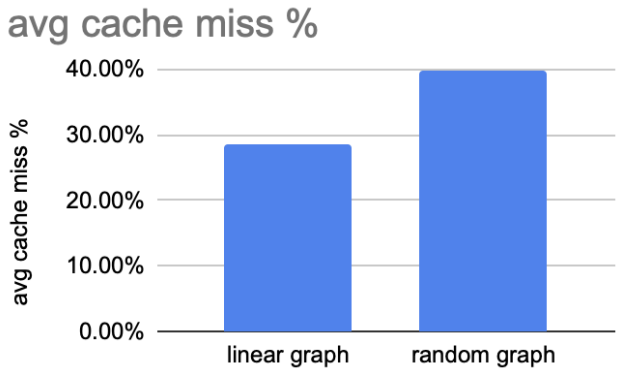
After the experiment, when the # of the nodes exceeding 300, the cache misses percentage has been rising sharply. So what I can say with this experiment, the size of more than 300 nodes exceeds the size of the cache. The other reason for the more nodes taking the more time is the more nodes make the higher cache-misses rate.

4. Checking Branch Predictability

In this experiment, I use Deque based BFS, but the type of the graph used is different. The random graph vs linear graph. Linear graph is like 1-2-3-4-5, which seems pretty easy to predict. The number of nodes is 10,000.



As I expected the linear graph shows less branch miss percentage. This is because the random graph has "if (std:: rand() % 2 ==0)" inside the main function and this line causes the branch miss prediction. The branch predictor is already good as general usage, but when the probability becomes 50-50 then, its performance goes poor.



What actually surprised me was the percentage of the cache-misses. I've never thought that there would be a huge cache miss % gap between the two. But, there was around a 30% gap! I can infer that better predictability can lead to efficient cache usage.

CONCLUSION

These findings highlight the importance of choosing the appropriate data structure and optimization techniques based on the specific characteristics of the graph being processed. Deques offer flexibility and efficiency for smaller graphs, whilst linked lists may be more suitable for very large graphs. Additionally, SIMD optimizations can provide substantial performance gains in BFS implementations. Understanding and optimizing for memory locality and branch predictability are essential for maximizing BFS performance, particularly in scenarios involving large and complex graphs.

Works Cited

Base Codes: https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/
https://gist.github.com/ahmadyan/e36edca7366069168385543ae53ffb7a
Data Recording code(extract_time.py) was built by ChatGPT 4o.